

Operate the schema

Migrations as code · N+1 trap · AsNoTracking · ExecuteUpdateAsync ·
soft-delete filters · Repository / UoW

Bulk path

```
await context.Enrollments  
    .Where(e => e.EnrolledAt < cutoff)  
    .ExecuteUpdateAsync(s => s.SetProperty(e  
=> e.IsArchived, true));
```

Run PostgreSQL like a team on call

Session outcomes

By the end of this session + linked lab:

- Treat migrations as code—read `Up()` and `Down()` before apply
- Produce and eliminate the N+1 anti-pattern while SQL logs scroll
- Use `AsNoTracking()` deliberately on read-heavy API paths
- Ship mass changes with `ExecuteUpdateAsync` in one SQL statement
- Layer soft-delete policy with `HasQueryFilter` (+ Ignore safely)
- Take a defensible Repository / UoW stance for TMS (Tier 6 write-up)

Migrations stay in Git—not just pgAdmin tweaks

Schema-as-code workflow

```
bash
```

```
# 1. Change C# model / Fluent config
```

```
dotnet ef migrations add AddStudentGpa
```

```
# 3. READ generated Up() / Down() – non-negotiable
```

```
dotnet ef database update
```

Professional rule

Manual column edits without migration files forks every developer database—block that habit early.

Up() and Down()

- Up() applies additive / destructive DDL as designed
- Down() rewinds ONLY what that migration authored
- `dotnet ef database update PreviousMigration` walks Down chain—schema rollback ≠ data resurrect

Friday deploy caveat

Dropped column data does not magically return after Down()—backup narrative belongs in facilitator notes.

N+1—the silent scalability tax

Bad loop · 1 + N round trips

Anti-pattern

```
var students = await context.Students.AsNoTracking().ToListAsync();

foreach (var s in students) {

    await context.Enrollments.AsNoTracking()
        .CountAsync(e => e.StudentId == s.Id);

}
```

Why teach it?

With lazy-loading proxies off, this pattern **forces** per-row queries so SQL logs cannot lie.

Shaped rescue · Include / projection

```
C#  
  
var shaped = await context.Students.AsNoTracking()  
    .Include(s => s.Enrollments)  
    .ToListAsync();  
  
// Or project aggregates in one SELECT without loading graphs
```

Tracking versus read-only blistering throughput

AsNoTracking()

Read-only leaderboard

```
var top = await context.Students.AsNoTracking()  
  
    .Where(s => s.GPA >= 3.5m)  
  
    .OrderByDescending(s => s.GPA)  
  
    .ToListAsync();
```

Operational rule

Most TMS GET pipelines: explicit AsNoTracking. Track entities only on mutation arcs.

ExecuteUpdateAsync • one SQL mutation

Bulk archive pattern

EF Core ≥ 7

```
await context.Enrollments
    .Where(e => e.EnrolledAt < cutoff)
    .ExecuteUpdateAsync(
        s => s.SetProperty(e => e.IsArchived, true));
```

Honest caveat

Bypasses change tracker—in-memory aggregates stay stale unless reloaded deliberately.

Soft delete · policy via HasQueryFilter

Global filters · Ignore sparingly

Fluent

```
modelBuilder.Entity<Student>()  
    .HasQueryFilter(s => !s.IsDeleted);  
  
await context.Students.IgnoreQueryFilters() // admins only  
    .ToListAsync();
```

Repository • Unit of Work—when TMS earns ceremony

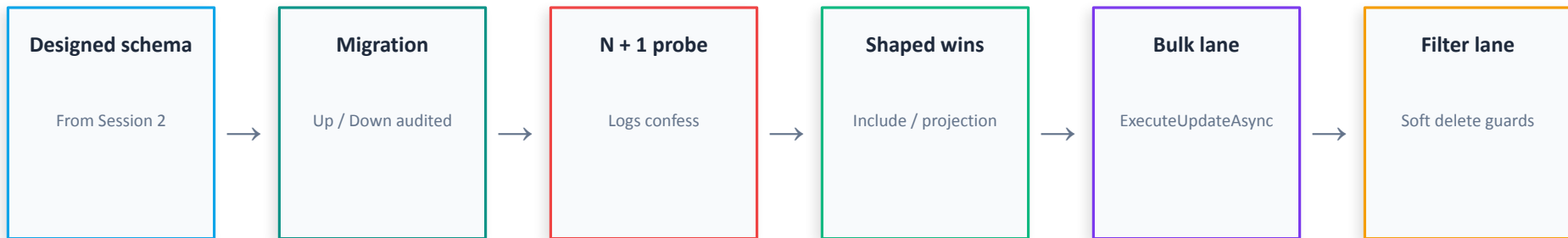
Position statement · not slogans

DbContext already implements Unit-of-Work; DbSet behaves like repositories for small APIs. Complexity arrives when enrolment workflows cross aggregates—invest abstraction then, not Week 1.

Tier 6 deliverable · 10-point design summary

Rubric expects...	You must cite...
Where Repo/UoW pays off	Concrete TMS flows spanning aggregates
Where Repo/UoW wastes time	Single-table lookups / naive CRUD
Tone	Position—hedging reads as incomplete ("both sides okay") fails

Blueprint · operating lane



Lab brief · Session 3

- Handout `**`M5-Lab-Session-3.md`**` — Exercises `**6 · 7 · 8 · 9**` plus Tier 6 summary
- Deliver: reviewed migrations · N+1 demo → fix · concurrency column · bulk archive respecting filters · authored stance
- Done when `**`M5-Lab-Session-3.md`` Lab Verification checklist`**` clears + artefact uploads

Tier 6 timing

Ship operations first—written summary only credible after SQL receipts exist.

M5 module synthesis

Thread	Carry into M6
IQueryable discipline	Controllers project through EF, not List<T> hacks
Designed graph	Navigation + payload already match TMS API stories
Operational rigor	Logs, bulk, filters become non-functional interview receipts

Interview cadence

Expect story beats: reading `Down()` before shared apply, detecting N+1 via logs, when `ExecuteUpdateAsync` bypasses tracker, why `IgnoreQueryFilters` is admin-only.

Next module · M6 REST surfaces

Controllers for Courses, Students, Enrollments on top of this persistence layer—contracts, status codes, Shape of JSON the world sees.